Check-in 2: Literature & Evaluation Design

Shahbaz Singh (0263901s)
COMP 5923 – Research in Computer Science
Instructor: Lydia Bouzar-Benlabiod
October 8, 2026

---

1 Research question

My question is still the same as in check-in 1: among RoPE, RMSNorm and the Muon optimizer, which ones lower validation loss compared to plain nanoGPT when added one at a time, and does that still hold at a bigger GPT-2 size?

2 Annotated bibliography

So far I have only skimmed these (abstract, intro and conclusion), so the notes below are about what each source is and why I picked it, not a deep reading.

[1] K. Jordan. modded-nanoGPT. GitHub repository, 2024. github.com/KellerJordan/modded-nanogpt

This is a speedrun project where people try to train a nanoGPT-style model as fast as possible, and it's where I got the idea for my project. RoPE, RMSNorm and Muon are all part of its list of changes. The tricks are added together as a stack, so it doesn't show how much each one helps on its own, which is the gap I want to look at.

[2] S. Narang et al. "Do Transformer Modifications Transfer Across Implementations and Applications?" *EMNLP*, 2021.

The authors tested many proposed changes to the transformer in the same codebase and found that most of them didn't really improve performance. This is why I don't want to assume every speedrun trick helps, and why I want to test them one at a time.

[3] J. Su et al. "RoFormer: Enhanced Transformer with Rotary Position Embedding." *Neurocomputing*, 2024.

This paper introduces RoPE, a different way of telling the model the position of each token. nanoGPT uses learned position embeddings, so RoPE would replace those. I'll use this paper as the reference when I implement it.

[4] B. Zhang, R. Sennrich. "Root Mean Square Layer Normalization." *NeurIPS*, 2019.

RMSNorm is a simpler version of LayerNorm. The paper says it works about as well as LayerNorm while being cheaper to compute. In my project it replaces LayerNorm in nanoGPT.

[5] K. Jordan et al. "Muon: An optimizer for hidden layers in neural networks." Blog post, 2024.

> This post explains the Muon optimizer that modded-nanoGPT uses instead of AdamW for most of the weights. It's a blog post, not a peer-reviewed paper, so I'm treating its claims with some caution.

3 Evaluation design (draft)

I haven't started building yet, so this is just my plan for now and some of it will probably change once I start running things.

The baseline is Karpathy's nanoGPT with its default Shakespeare-character settings, since it's small enough to train in a few minutes on Colab. On top of that there are three versions, each with only one change: one with RoPE, one with RMSNorm and one with Muon. That way if the loss changes, I know which trick caused it.

The main thing I'm comparing is validation loss, since nanoGPT already prints it during training. The loss curves are worth looking at too, because a trick might help the model learn faster even if it ends up at the same loss. All runs use the same data, the same number of steps and the same settings apart from the one change. Muon is a bit slower per step though, so I want to keep track of how long each run takes as well.

Each version gets at least 3 different seeds, because one run on its own could just be lucky. Then I can compare the average loss to the baseline and use a t-test to see if the difference is real or just normal variation between runs.

If there's still time and budget left (around $20–50), I'd like to try the tricks that helped on a small GPT-2 model, but with fewer runs.

4 Things I'm not sure about yet

- Muon probably needs a different learning rate than AdamW. I'm thinking of using the value from modded-nanoGPT, but then the baseline's learning rate wasn't tuned in the same way, which might not be fair.
- I'm not sure if 3 seeds is enough to see small differences.
- I'm not sure if "same number of steps" is the right way to match compute for Muon, or if I should use training time instead.